

Processes: Process Environment and Control

Brianna and Alena

AGENDA

2

01 fork

- pids
- fork diagrams
- copied memory

02 waitpid

03 exec

04 code review

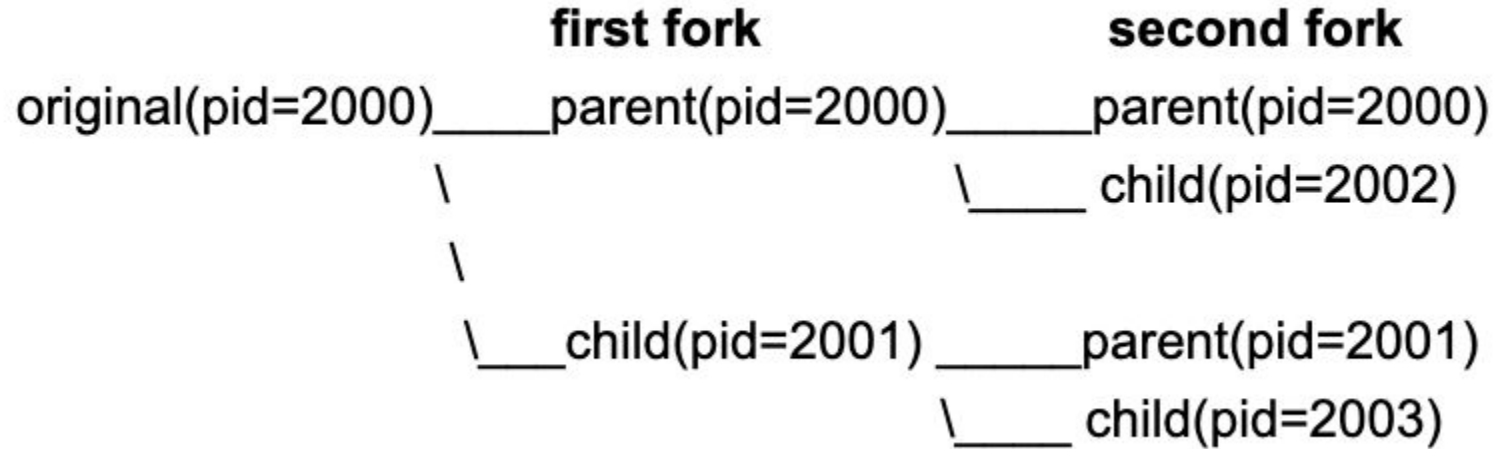
pid_t fork()

- purpose: create a child process
- return value:
 - parent process: child pid (> 0)
 - child process: 0
 - error: < 0
- child resumes execution after fork() call

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

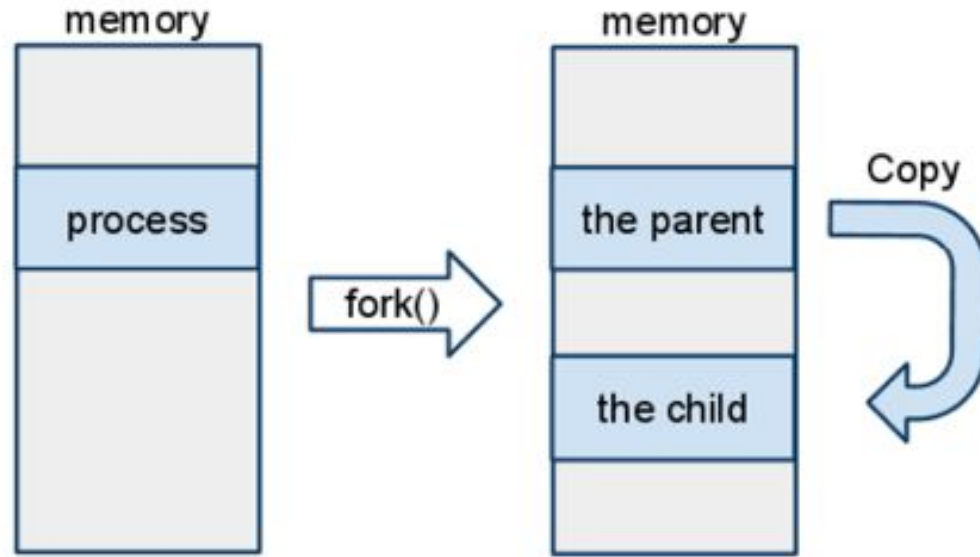
int main()
{
    pid_t pid;

    if ((pid = fork()) < 0) {
        perror("fork");
        return -1;
    } else if (pid == 0) {
        /* child */
        printf("I am the child\n");
        return 0;
    }
    /* parent */
    printf("I am the parent of %d\n", pid);
    return 0;
}
```



Copied Memory When Forking

- child is copy of parent, receives a copy of the parent's:
 - data space
 - heap
 - stack
- parent and child share text segment



waitpid

```
pid_t waitpid(pid_t pid, int *status, int options);
```

pid_t pid:

- < -1: wait for any child process whose process group ID = absolute value of pid
- -1: wait for any child process
- 0: wait for any child process whose process group ID = that of calling process
- > 0: wait for child whose process ID = value of pid

waitpid

pid_t waitpid(pid_t pid, int *status, **int options**);

int options:

- 0: (default), only waits for terminated children
- WNOHANG: return immediately if no child has exited
- WUNTRACED: return if a child has stopped
- WCONTINUED: return if a stopped child has been resumed by delivery of SIGCOUNT

exec

```
int exec(const char *pathname, const char *argv, ..., NULL);
```

- completely replaces program of a process with the new program (new program begins executing at main function)
 - also replaces text, data, heap, and stack segments with new program
- PID doesn't change (new process isn't created)

exec

```
int exec(const char *pathname, const char *argv, ..., NULL);
```

const char *pathname / filename

- if filename contains slash, taken as a pathname
- the path to the program (executable) that you want to replace the current process
- can be absolute or relative path
 - /bin/lis
 - ./program

exec

```
int exec(const char *pathname, const char *argv, ..., NULL);
```

const char *argv, ..., NULL

- passing in command-line arguments to the new program
- first element typically name of program being executed (typically same as pathname for convention, but doesn't have to be)
- mark end of argument with NULL pointer
- if using 0, must cast to char *
 - (char *)0

code
review

```

/**
 * Determines whether or not the input string represents a valid integer.
 * A valid integer has an optional minus sign, followed by a series of digits
 * [0-9].
 */
bool is_integer(char *input) {
    int start = 0, len = strlen(input);

    if (len >= 1 && input[0] == '-') {
        if (len < 2) {
            return false;
        }
        start = 1;
    }
    for (int i = start; i < len; i++) {
        if (!isdigit(input[i])) {
            return false;
        }
    }
    return len > 0;
}

/**
 * Takes as input a string and an in-out parameter value.
 * If the string can be parsed, the integer value is assigned to the value
 * parameter and true is returned.
 * Otherwise, false is returned and the best attempt to modify the value
 * parameter is made.
 */
bool get_integer(char *input, int *value) {
    long long long_long_i;
    if (sscanf(input, "%lld", &long_long_i) != 1) {
        return false;
    }
    *value = (int)long_long_i;
    if (long_long_i != (long long)*value) {
        fprintf(stderr, "Warning: Integer overflow with '%s'.\n", input);
        return false;
    }
    return true;
}

```

long long guarantees 64 bits on all systems (in contrast to just long)

```

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <num forks>\n", argv[0]);
        return EXIT_FAILURE;
    }
    int num_forks = 0;    num_forks: number of forks to attempt (command-line input)
    if (!is_integer(argv[1]) || !get_integer(argv[1], &num_forks) || get_integer stores argv[1] into int num_forks
        num_forks <= 0) {
        fprintf(stderr, "Error: Invalid number of forks '%s'.\n", argv[1]);
        return EXIT_FAILURE;
    }

    /*
     * TODO:
     * Use fork() and execl().

     Create num_forks children processes.
     If any call to fork fails, an error message should be printed and
     the loop should continue.
     Keep track of how many children were actually spawned.
     The error message will be of the form:
     Error: fork[fork_index] failed. %s.
     fork_index will be an integer [1, ..., num_forks]
     %s will be strerror(errno)

     The children should execute randomsleep with no arguments.
     If execl fails, an error message should be printed and
     EXIT_FAILURE should be returned.
     The error message will be of the form:
     Error: execl() failed for pid %ld. %s.
     %s will be strerror(errno)
     */

    pid_t pid;
    int num_started = 0;    num_started: var to count actual number of forked child processes
    for (int i = 1; i <= num_forks; i++) {
        if ((pid = fork()) < 0) {
            fprintf(stderr, "Error: fork[%d] failed. %s.\n", i,
                strerror(errno));
            continue;
        } else if (pid == 0) {
            if (execl("randomsleep", "randomsleep", NULL) == -1) {
                fprintf(stderr, "Error: execl() failed for pid %ld. %s.\n",
                    (long) getpid(), strerror(errno));
                return EXIT_FAILURE;
            }
        } else {
            num_started++;
        }
    }

    /*
     * TODO:
     * Use wait().

     We are in the parent here. Wait for all children that were actually
     spawned. As each child terminates, print the message:
     Child with pid <pid> has terminated.
     If an error occurs during waiting, instead print the message:
     Error: wait() failed. %s.
    */
}

```

child leaves, and if execl is
successful, doesn't go further

```
    %s will be strerror(errno)
*/
for (int i = 0; i < num_started; i++) {
    pid_t child_pid = wait(NULL);
    if (child_pid < 0) {
        fprintf(stderr, "Error: wait() failed. %s.\n", strerror(errno));
    } else {
        printf("Child with pid %ld has terminated.\n", (long)child_pid);
    }
}
printf("All children are done sleeping.\n");
return EXIT_SUCCESS;
}
```

wait(NULL) blocks until any of the child processes are terminated, and NULL means don't store the child's exit status (the parent doesn't care about the child's exit status)

```
(base) alenachan@Alenas-MacBook-Air-3 nforks % ./nforks 3
I, pid 87473, am going to sleep for 1.79 seconds.
I, pid 87474, am going to sleep for 3.77 seconds.
I, pid 87475, am going to sleep for 1.23 seconds.
I, pid 87475, have awakened.
Child with pid 87475 has terminated.
I, pid 87473, have awakened.
Child with pid 87473 has terminated.
I, pid 87474, have awakened.
Child with pid 87474 has terminated.
All children are done sleeping.
(base) alenachan@Alenas-MacBook-Air-3 nforks %
```

THANKS FOR
LISTENING